

UNIVERSIDAD DIEGO PORTALES



FACULTAD DE INGENIERÍA Y CIENCIAS

TAREA 2: ANÁLISIS DEL PROTOCOLO RTMP BAJO INFRAESTRUCTURA DE CONTENEDORES

Estudiantes:
Jeremías Quezada
Matías Droguett

Capítulo 1: Introducción e Investigación del Protocolo

1.1 Marco Teórico y Objetivo Principal del Protocolo RTMP

El protocolo **RTMP** (Real-Time Messaging Protocol) es un protocolo de la capa de aplicación, desarrollado originalmente por Macromedia (actualmente Adobe Systems). Su **objetivo principal** es facilitar la transmisión síncrona, persistente y de baja latencia de flujos multimediales (audio, video y metadatos de control) a través de redes IP.

A diferencia de los paradigmas tradicionales de transferencia de archivos basados en el protocolo HTTP, donde el cliente debe solicitar constantemente segmentos estáticos mediante peticiones asíncronas, RTMP establece un canal virtual directo y continuo. Esto permite que los datos multimedia se transmitan instantáneamente en ráfagas de tiempo real, minimizando de forma drástica el retardo intrínseco (*delay*) de la red y optimizando la entrega para entornos altamente dinámicos.

1.2 Justificación Tecnológica: ¿Por qué y cuándo utilizar RTMP?

Comprender la viabilidad de RTMP requiere analizar sus ventajas operativas y sus casos de uso específicos dentro de las arquitecturas de red actuales:

¿Por qué utilizarlo? (Ventajas Técnicas)

- **Conexión Persistente Única:** Al operar sobre una única conexión de transporte, evita la sobrecarga (*overhead*) de negociación y los saludos de tres vías repetitivos de TCP para cada fragmento de video.
- **Mecanismo de Multiplexación y Chunking:** RTMP segmenta los mensajes en bloques pequeños denominados *Chunks*. Al entrelazar fragmentos de audio, video y control en un único canal lógico, previene que los paquetes grandes de video bloqueen los paquetes de audio o los mensajes críticos de control.
- **Baja Latencia Nativa:** Permite transmisiones con retardos de extremo a extremo que oscilan entre los 3 y 5 segundos, superando con creces los 10 o 30 segundos característicos de los protocolos basados en archivos HTTP tradicionales como HLS básico.

¿Cuándo utilizarlo? (Casos de Uso en la Industria) En el ecosistema moderno de streaming, RTMP se ha consolidado firmemente en la fase de **Contribución o Ingesta** de video, mas no en la fase de distribución masiva al usuario final.

- **Ingesta hacia Servidores de Streaming (Streaming Uplink):** Es el estándar absoluto utilizado por softwares codificadores locales (como OBS Studio o la suite FFmpeg) para inyectar flujos de video en vivo hacia las plataformas de distribución global (e.g., Twitch, YouTube Live o servidores perimetrales de CDNs).
- **Sistemas de Monitoreo e Interactividad de Baja Latencia:** Se emplea en aplicaciones de videovigilancia de circuito cerrado y salas de control centralizadas, donde se requiere una respuesta visual inmediata ante eventos físicos remotos.

1.3 Descripción Detallada del Tráfico Esperado

Dado que RTMP se apoya sobre el protocolo de transporte orientado a la conexión **TCP** a través del puerto bien conocido **1935**, el comportamiento del tráfico capturado en un análisis de red promiscuo exhibe tres fases secuenciales claramente distinguibles:

- **Fase de Handshake Estricto (C0/C1/C2 y S0/S1/S2):** Secuencia síncrona obligatoria donde el cliente envía un byte crudo de versión (**C0**) y un bloque de 1528 bytes de marcas de tiempo (**C1**). El servidor responde en el mismo formato (**S0+S1+S2**) y el cliente confirma con **C2**. Este proceso valida la sincronía temporal inicial y establece los cimientos para el tamaño de bloque dinámico.
- **Fase de Negociación y Mensajes de Control (AMF0):** Tráfico serializado bajo el formato *Action Message Format* (AMF0). En esta fase es posible leer en texto claro comandos lógicos de señalización como **connect** (inicialización de sesión), **publish** (solicitud del emisor para inyectar datos) y **play** (solicitud del receptor para consumir datos).

- **Fase de Flujo Multimedia (Chunks):** Inundación masiva y constante de ráfagas de datos etiquetadas por los analizadores como *Audio Data* (codificado en AAC) y *Video Data* (codificado en H.264). Estos paquetes viajan prevalentemente con la bandera TCP PSH (*Push*) activa para forzar al núcleo del sistema operativo a procesar y renderizar los fragmentos inmediatamente sin retenerlos en buffers locales del sistema de transporte.

1.4 Guía de Instalación del Motor de Contenedores (Docker)

Para el despliegue controlado de la topología, el sistema anfitrión requiere el motor de Docker. En sistemas basados en la distribución Ubuntu/Debian, el aprovisionamiento se realiza mediante los repositorios oficiales usando la terminal:

```
# 1. Actualizar el índice de paquetes local del sistema
sudo apt-get update

# 2. Instalar paquetes prerequisites para permitir el uso de repositorios sobre HTTPS
sudo apt-get install -y ca-certificates curl gnupg

# 3. Descargar e incorporar la llave GPG oficial de Docker para validación criptográfica
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
  ↪ /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# 4. Configurar el repositorio estable de Docker en las fuentes de APT
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
  ↪ https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
  ↪ stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 5. Instalar el motor de Docker, la interfaz de comandos (CLI) y el contenedor runtime
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

1.5 Configuración de Archivos Dockerfile (Infrastructure as Code)

Bajo el paradigma de IaC, se estructuraron dos recetas locales independientes dentro del directorio raíz del proyecto ('tarea2-redes/'), permitiendo compilar imágenes personalizadas y reproducibles sin depender de repositorios externos mutables.

A. Dockerfile del Servidor ('./server/Dockerfile') La configuración del nodo central hereda de una imagen base oficial de Node.js e instala el servidor multimedia ligero:

```
FROM node:18-alpine
WORKDIR /usr/src/app
# Instalar de manera global el paquete que levanta el servidor RTMP
RUN npm install -g node-media-server
# Exponer el puerto de streaming RTMP (1935) y el panel de administración web (8000)
EXPOSE 1935 8000
CMD ["node-media-server"]
```

B. Dockerfile del Emisor ('./client/Dockerfile') La configuración del cliente inyector se apoya en una base alpina optimizada con las herramientas binarias de codificación de la suite FFmpeg:

```
FROM alpine:3.19
# Instalar software FFmpeg y dependencias de codecs multimedia libres
RUN apk add --no-cache ffmpeg
# Comando de automatización por defecto: inyecta barras de prueba y tono senoidal al arrancar
CMD ffmpeg -re -f lavfi -i "testsrc=size=640x480:rate=30" -f lavfi -i
  ↪ "sine=frequency=1000:sample_rate=44100" -c:v libx264 -c:a aac -f flv
  ↪ rtmp://rtmp-server/live/prueba
```

1.6 Documentación de Configuración y Despliegue de la Topología

A continuación, se detallan los comandos ordenados cronológicamente para inicializar, configurar y desplegar la infraestructura de red virtualizada.

A. Configuración de Entorno y Red Virtual Aislada Se incorpora un paso de limpieza preventiva para mitigar errores de colisión por redes remanentes en el demonio de Docker, garantizando un despliegue transparente:

```
# Eliminación preventiva de la red si ya existía de ejecuciones previas
sudo docker network rm red-rtmp 2>/dev/null || true

# Creación de la red puente dedicada para el análisis cerrado de RTMP
sudo docker network create red-rtmp
```

B. Compilación Local de Componentes (Aprovisionamiento IaC) La construcción de las aplicaciones se realiza invocando los Dockerfiles locales:

```
# Compilación de la imagen del servidor local RTMP
sudo docker build -t rtmp-server-local ./server

# Compilación de la imagen del emisor multimedia automatizado
sudo docker build -t rtmp-emisor-local ./emisor
```

C. Despliegue de Servicios y Generación de Tráfico El orden de encendido de los contenedores y herramientas es crítico para asegurar que los flujos de control sean capturados correctamente:

1. Instanciación y ejecución del Servidor RTMP:

```
sudo docker run -d --name rtmp-server --network red-rtmp -p 1935:1935 -p 8000:8000
↪ rtmp-server-local
```

2. **Hito de Captura en Wireshark:** En este punto exacto, antes de activar el emisor, se inicia la escucha promiscua en Wireshark seleccionando la interfaz virtual puente generada por Docker, garantizando registrar el protocolo desde su estado base.

3. Activación del Emisor Multimedia Automatizado:

```
sudo docker run -d --name rtmp-emisor --network red-rtmp rtmp-emisor-local
```

4. Consumo del flujo en tiempo real mediante el cliente de consola desde el sistema anfitrión:

```
ffplay rtmp://127.0.0.1/live/prueba
```

Capítulo 2: Análisis de Tráfico

2.1 Tráfico Observado al utilizar los softwares instalados

Al inspeccionar detenidamente las tramas capturadas en la interfaz de red virtual empleando **Wireshark** y aplicando el filtro dedicado **rtmpt** (o la regla de aislamiento de transporte **tcp.port == 1935**), se validaron empíricamente las hipótesis planteadas en el Capítulo 1:

1. **Establecimiento y Handshake:** Quedó registrado el saludo de tres vías de TCP, seguido inmediatamente por los bloques binarios de sincronización RTMP. Se observó el envío de los paquetes **C0+C1** desde la dirección IP del emisor (**172.18.0.3**) hacia el servidor (**172.18.0.2**), la réplica síncrona del servidor a través de las tramas **S0+S1+S2** y la consolidación de la línea de tiempo a través del paquete final **C2**.
2. **Comandos AMF0 en Texto Claro:** El analizador parseó con éxito las estructuras jerárquicas del protocolo de control. Quedó documentado el momento preciso en que el emisor efectúa la llamada RPC remota **publish** apuntando a la cadena de flujo **"prueba"**. De igual forma, al arrancar la herramienta de consola local **ffplay**, se interceptó el comando **play** solicitando el consumo del mismo identificador, lo que gatilló que el servidor emitiera el paquete de estado confirmatorio **NetStream.Play.Start**.
3. **Flujo Continuo y Chunks Multimedia:** Una vez aceptada la sesión, la captura se satura con paquetes masivos continuos del canal lógico indexados como *Video Data* y *Audio Data*. Estos datos viajan empaquetados bajo tramas TCP con la bandera **PSH** activa, forzando la entrega inmediata al espacio de usuario para su renderizado fluido.

2.2 Suposiciones sobre posibles repercusiones (Modificación On-the-fly)

Debido a la naturaleza binaria rígida y fuertemente tipada de las cabeceras de los bloques de RTMP, alterar la estructura interna de los paquetes en tránsito produce un colapso catastrófico inmediato en las aplicaciones consumidoras.

Suposición de Fallo: Modificación del campo *Message Length* en un Chunk Multimedia.

Si un agente malicioso interceptara el tráfico de la red e inyectara una modificación arbitraria en el valor numérico del campo *Message Length* embebido en la cabecera de un Chunk RTMP antes de que este arribe al socket de escucha de la aplicación cliente, se corrompería la máquina de estados del deserializador.

Repercusiones en el Software: De manera análoga a lo que ocurriría con la herramienta *ping* si alteráramos de manera dinámica el campo *Type* de la cabecera de un paquete ICMP (el sistema operativo anfitrión perdería el contexto lógico, siendo incapaz de determinar si procesar un *Echo Request* o un *Echo Reply*, procediendo a descartar la trama), el reproductor **FFplay** depende críticamente del campo *Message Length*. Este valor le indica al software cuántos bytes exactos debe extraer del buffer de red para reensamblar y reconstruir un fotograma de video o una muestra de audio comprimida en la memoria RAM.

Si el tamaño binario declarado en la cabecera adulterada no coincide con la carga útil real (*payload*) entregada por la capa TCP, el puntero de lectura de memoria del decodificador H.264 se desalineará por completo, forzando al programa a interpretar bytes de video como si fuesen metadatos o direcciones de memoria basura.

A nivel práctico y visual, esta corrupción repercutirá secuencialmente en tres etapas críticas: primero, la ventana de reproducción perderá la sincronía de los cuadros clave (I-Frames), mostrando severos artefactos verdes en la pantalla y píxeles completamente rotos o congelados; segundo, el flujo de audio sufrirá ruidos ásperos y desincronizaciones masivas (*audio glitches*); y tercero, al persistir la lectura errónea fuera de los límites lógicos de memoria asignados al búfer, el software sufrirá un desbordamiento de búfer (*Buffer Overflow*) o un fallo de segmentación (*Segmentation Fault*), provocando el cierre forzado e inesperado de la ventana gráfica en la computadora del usuario (*Crash*).

Conclusiones

El desarrollo práctico de este laboratorio permitió afianzar conceptos fundamentales sobre redes de transmisión multimedia y la virtualización avanzada de arquitecturas de servicios. Las conclusiones lógicas más relevantes del experimento corresponden a:

- **Mecanismo de Enmascaramiento y Enrutamiento en Docker:** Se corroboró empíricamente que el motor de contenedores Docker realiza un proceso de traducción y enmascaramiento de red mediante su controlador tipo puente (*bridge*). A pesar de que el cliente local apunte sus peticiones a la dirección de loopback local **127.0.0.1** en el host, a nivel de captura en Wireshark dentro del puente virtual, el tráfico es traducido y mapeado asignándole al host la dirección IP correspondiente a la puerta de enlace predeterminada del segmento aislado (**172.18.0.1**).
- **Condición de Carrera y Pérdida de Estado en el Analizador:** Se descubrió la dependencia temporal crítica existente en el análisis promiscuo de protocolos con estado. Si la herramienta de captura Wireshark se activa de forma tardía respecto al encendido del emisor, se pierde el registro de los paquetes binarios iniciales del Handshake y la negociación de control AMF0. Dado que en esos primeros paquetes se define dinámicamente el tamaño de fragmento compartido (**Set Chunk Size**), al carecer de este contexto histórico, el motor de análisis de Wireshark es incapaz de decodificar la fragmentación del canal lógico, viéndose forzado a catalogar erróneamente todo el tráfico multimedia subsiguiente de manera genérica como paquetes de datos **Unknown** o TCP crudo.

Referencias

- Adobe Systems Incorporated. (2012). *Real-Time Messaging Protocol (RTMP) Specification*. v1.0.
- Node-Media-Server Project. (2026). *Documentation and Container Architectures for Node.js Media Servers*.

Anexos

Se listan los elementos gráficos requeridos para la validación visual de la experiencia:

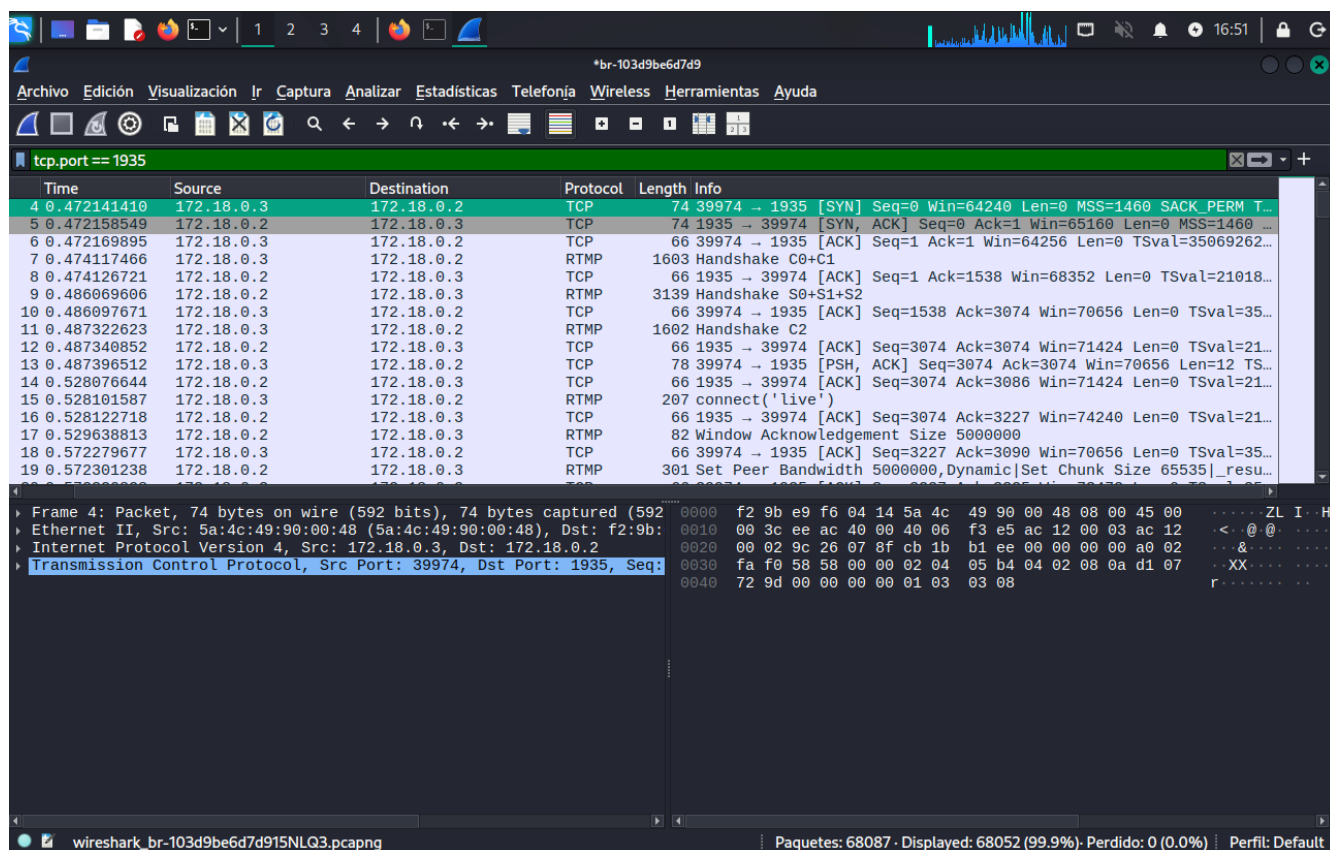


Figura 1: **Figura 1:** Captura en Wireshark del Handshake inicial exitoso (paquetes binarios C0, C1, S0, S1, S2) filtrado bajo el puerto 1935.

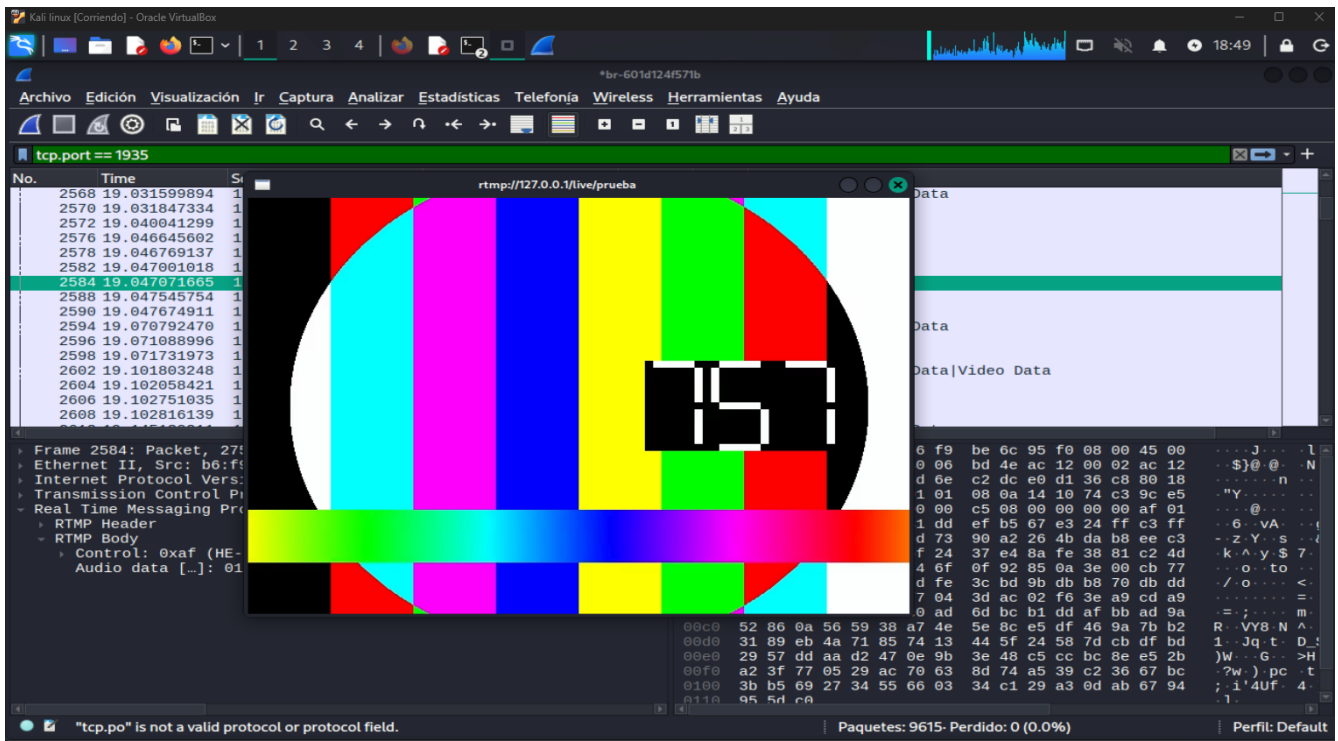


Figura 2: **Figura 2:** Ejecución interactiva del comando `ffplay` en terminal mostrando la ventana gráfica renderizando el patrón de pruebas multimedia de barras de colores.

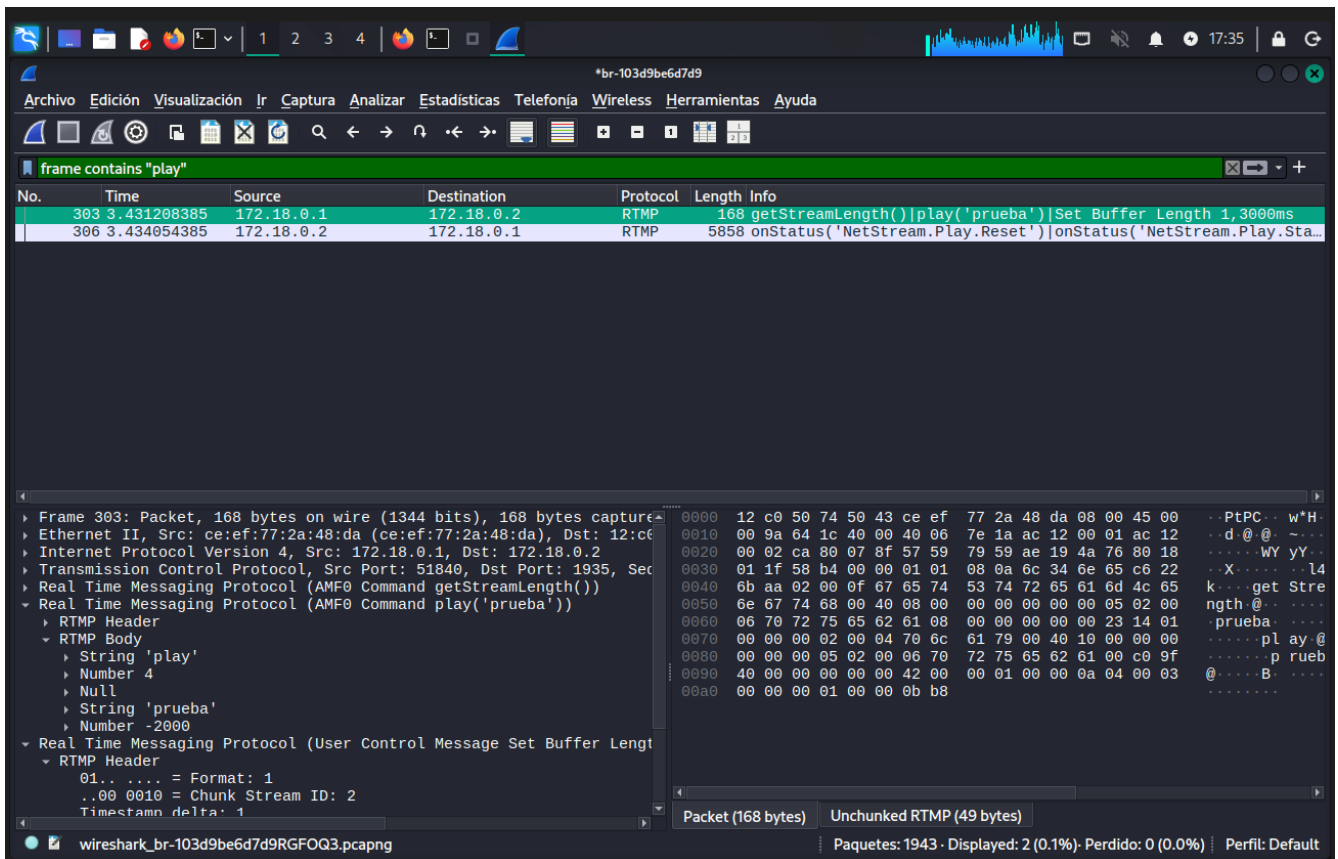


Figura 3: Figura 3: Análisis en Wireshark del comando AMF0 **publish** enviado por el emisor y del comando de consumo **play** interceptado en la interfaz de red local.